Result

Output

dp1.c

N: 1000000 <T>: 0.000908 sec B: 8.813 GB/sec F: 2203316508.243 FLOP/sec

N: 300000000 <T>: 0.281464 sec B: 8.527 GB/sec F: 2131707833.537 FLOP/sec

dp2.c

N: 1000000 <T>: 0.000240 sec B: 33.267 GB/sec F: 8316857984.202 FLOP/sec

N: 300000000 <T>: 0.133739 sec B: 17.945 GB/sec F: 4486365082.982 FLOP/sec

dp3.c

N: 1000000 <T>: 0.000070 sec B: 114.135 GB/sec F: 28533814481.927 FLOP/sec

N: 300000000 <T>: 0.023646 sec B: 101.495 GB/sec F: 25373780741.202 FLOP/sec

dp4.py

N: 1000000, <T>: 0.343628 sec, B: 0.023281 GB/sec, FLOPS: 5820247.513 FLOP/sec

N: 300000000, <T>: 105.541696 sec, B: 0.022740 GB/sec, FLOPS: 5684956.976 FLOP/sec

dp5.py

N: 1000000, <T>: 0.000107 sec, B: 74.484630 GB/sec, FLOPS: 18621157500.311 FLOP/sec

N: 300000000, <T>: 0.100413 sec, B: 23.901358 GB/sec, FLOPS: 5975339620.393 FLOP/sec

Q1

By using the second half of the measurements, it can improve accuracy and the stability of consequence. For example, cache may not be fully utilized in the first half part.

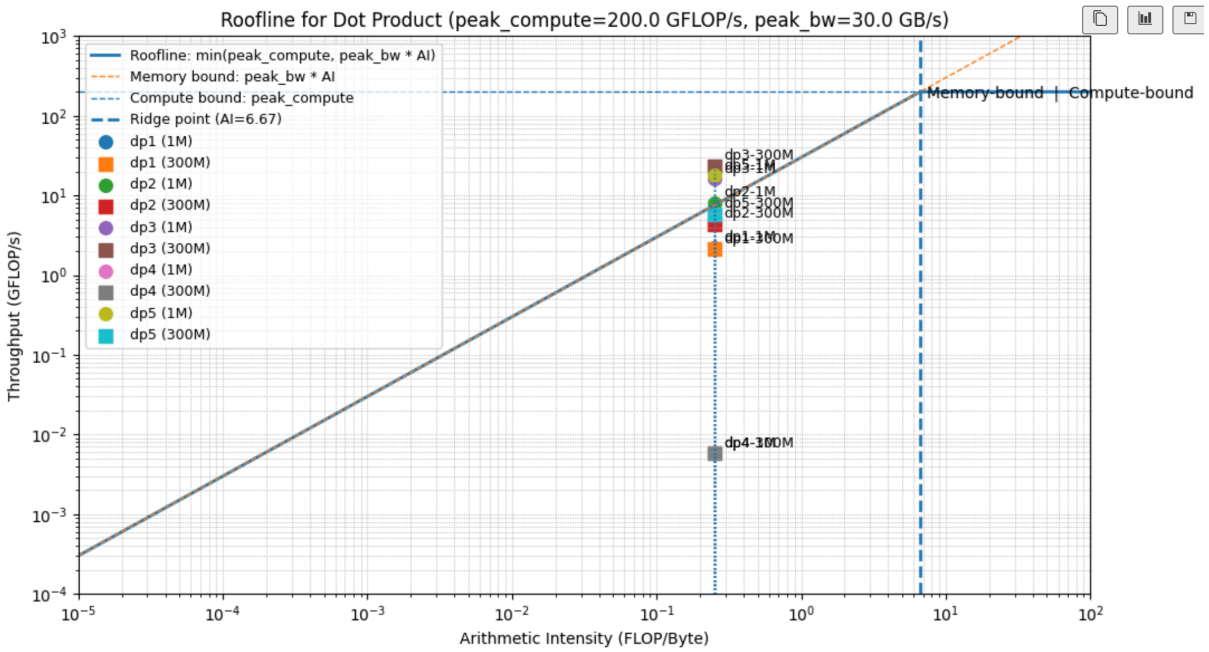
Time is additive data, so it should use arithmetic mean.

Q2

Performance

Group	BW_avg(GB/s)	TP_avg(GFLOP/s)	AI(F/B)
dp1-1M	8.816667	2.204109	0.249993
dp1-300M	8.526333	2.131534	0.249994
dp2-1M	32.438000	8.109558	0.250002
dp2-300M	17.628667	4.407153	0.249999
dp3-1M	64.756000	16.189066	0.250001
dp3-300M	93.277000	23.319282	0.250000
dp4-1M	0.023129	0.005782	0.249998
dp4-300M	0.023053	0.005763	0.249998
dp5-1M	71.348646	17.837161	0.250000
dp5-300M	23.439850	5.859963	0.250000

Roofline



They are all memory bind because they are in the left of ridge point (arlthmetic intensity = 6.6).

From the picture above, we can see that the arithmetic intensity is very close, which is due to the algorithm of dot product. In dot product, it need to load 2N data and calculate ~2N times. Therefore, arithmetic intensity is a specific value.

$$AI = \frac{2N}{2N * sizeof(float)} = 0.25$$

Underperform tasks:

- dp1 & dp2
  - Because it only loads 2 or 8 data per loop, so it need to access memory many times.
- dp4
  - DP4 is extremely slow because python is an interpreted language. The interpreter need to do a lot of work in each loop, and the data reading is as quick as C.

Q3

Using the N = 300000000 simple loop as the baseline, the performance are as below:

Group	BW_avg(GB/s)	TP_avg(GFLOP/s)
dp1-300M	8.526333	2.131534
dp2-300M	17.628667	4.407153
dp3-300M	93.277000	23.319282
dp4-300M	0.023053	0.005763
dp5-300M	23.439850	5.859963

- DP2 is faster than DP1 because it can read more data in one loop.
- DP3 is much faster than DP1 and DP2 because MKL leverage many mechanism to accelerate calculating, such as SIMD vectorization and

fused multiply-add.

- DP4 is extremely slow mainly because the overhead of Python interpreter.
- DP5 is faster than DP4 because it does not use python loop. Instead, it calls highly optimized library like MKL.

Q4

I find that the dot product result of DP1-300M and DP2-300M are wrong.

	Actual result	Theoretic result
DP1	16777216 = 2^24	300000000
DP2	67108864 = 2^26	300000000

In each loop of DP1, result = result + 1. However, when performing floating-point number operations:

$$2^{24} + 1 = 2^{24}$$

That is because a float32 only has 23 bits to store the mantissa part. When transforming 1 to a form like $n * 2^{24}$, the value is lost due to precision issue.

The issue in DP2 is the same as in DP1, since both suffer from floating-point precision limitations during accumulation.